Folleto del estudiante Encuentro 3: ORM, generación de reportes y consumo de servicios web

Departamento de Ciencias, Tecnología y Salud – CUR Matagalpa

Fecha: ___/___/2026

UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA, MANAGUA
UNAN-MANAGUA

UNIVERSIDAD EN EL campo

Universidad Nacional Autónoma de Nicaragua, Managua · Centro Universitario Regional de Matagalpa · Departamento de Ciencias, Tecnología y Salud

Programación de Aplicaciones de Escritorio – Folleto del Encuentro 3: ORM, generación de reportes y consumo de servicios web

Heydy Judelki Mairena Palacios

Ingeniería en Sistemas · II año – II semestre 2026 · Unidad(es) IV, V del programa

Fecha de emisión: ___/___/2026

¡Universidad del Pueblo y para el Pueblo!

Elaborado por:	Revisado por:	Autorizado por:
Heydy Judelki Mairena Palacios	⚠ PENDIENTE (coordinador/a de carrera)	⚠ PENDIENTE (director/a de departamento)
Perfil del autor (opcional): ⚠ completar si se desea.		

Presentación

Este módulo cierra el proyecto integrador. AgroStock dejará de escribir SQL a mano gracias a un ORM, generará reportes de inventario en PDF y Excel, y consultará en Internet el tipo de cambio del córdoba para valorar el inventario en dólares. Se abordan las Unidades IV y V del programa y se integran todos los aprendizajes del semestre.

Proyecto integrador: AgroStock: aplicación de escritorio para el inventario de un agroservicio o cooperativa de la zona (productos, categorías, existencias, reportes y tipo de cambio). Se construye por etapas: v1 interfaz gráfica (E1) → v2 base de datos SQLite + importación concurrente (E2) → v3 ORM, reportes PDF/Excel y consumo de servicio web (E3).

Horas del programa para este módulo: HT 10 · HP 20 · HTI 15 (total 45 h).

Competencias y resultados de aprendizaje

Competencias del programa a las que contribuye este módulo:

- Desarrolla aplicaciones de escritorio con interfaz gráfica que acceden a bases de datos, implementan programación concurrente (hilos), utilizan ORM, generan reportes y consumen servicios web; mediante la aplicación de un lenguaje de programación orientado a objetos para resolver problemas de la vida cotidiana.
- Aplica procesos creativos e investigativos haciendo uso de medios tecnológicos en el desarrollo de sus aprendizajes.
- Demuestra actitudes de respeto hacia las personas y los saberes ancestrales, inclusión y equidad para una cultura de paz en su entorno.
- Cuida la Madre Tierra como ser vivo y sujeto de dignidad en la convivencia diaria con el entorno natural y los ecosistemas, con el fin de promover su regeneración y asegurar el bien común supremo para todas las especies y generaciones futuras.

Resultados de aprendizaje:

- Desarrolla aplicaciones de escritorio con interfaz gráfica que acceden a bases de datos mediante el uso de ORM y que generan reportes.
- Desarrolla aplicaciones de escritorio con interfaz gráfica que consumen servicios web.

Objetivos del encuentro:

- Utilizar un ORM (Entity Framework Core) para realizar operaciones CRUD sin escribir SQL manualmente y generar reportes en PDF y Excel a partir de datos reales.
- Consumir servicios web RESTful desde una aplicación de escritorio, procesando respuestas JSON o XML y manejando fallos de conexión.

Contenidos

Unidad IV. Uso de ORM y generación de reportes: Uso de ORM para realizar operaciones CRUD sobre una base de datos; Uso de herramienta para la generación de reportes.

Unidad V. Consumo de servicios web: Clases para el consumo de servicios web; Estudios de caso para el consumo de servicios web.

I. Entrada temática – Vivencia

La cooperativa a la que pertenece el agroservicio de doña Martha pide cada lunes un reporte de inventario en PDF y el valor total en dólares con el tipo de cambio del día. Su hijo copia a mano la tabla en Excel y busca el tipo de cambio en el celular; si la señal es mala, el reporte se atrasa.

1. ¿Qué partes de este proceso podría automatizar AgroStock?
2. En AgroStock v2 escribiste muchas sentencias SQL parecidas. ¿Qué pasaría si mañana hubiera que cambiar a SQL Server?
3. ¿Qué debería hacer la aplicación cuando no hay Internet?
4. ¿Cuánto papel y cuántos viajes se ahorran si el reporte se envía en digital?

> Lo que esperamos aprender: manipular la base de datos con objetos (ORM), producir reportes profesionales y conectar la aplicación con servicios en Internet de forma robusta.

II. Desarrollo central – Fundamentación científica

Unidad IV · 1. ¿Qué es un ORM?

Un ORM (Object-Relational Mapper, mapeador objeto-relacional) traduce automáticamente entre los objetos del programa y las tablas de la base de datos. El programador trabaja con clases y colecciones; el ORM genera el SQL, abre las conexiones y convierte las filas en objetos. En .NET, el ORM oficial es Entity Framework Core (EF Core).

Mundo orientado a objetos	Mundo relacional
Clase Producto	Tabla Productos
Propiedad Nombre	Columna Nombre
Objeto (instancia)	Fila
Referencia Producto.Categoria	Clave foránea CategoriaId
List<Producto> Categoria.Productos	Relación 1:N

- Ventajas: menos código repetitivo, consultas verificadas por el compilador (LINQ), independencia del motor (SQLite, SQL Server, MySQL, PostgreSQL), protección contra inyección SQL.
- Límites: menos control sobre el SQL generado; consultas muy complejas pueden ser menos eficientes; hay que entender qué consulta se ejecuta y cuándo.

Unidad IV · 2. Entity Framework Core: entidades y DbContext

```
public class Producto
{
    public int Id { get; set; }                    // clave primaria por convención
    [Required, MaxLength(10)] public string Codigo { get; set; } = "";
    public int CategoriaId { get; set; }           // clave foránea
    public Categoria? Categoria { get; set; }      // propiedad de navegación
    public decimal Precio { get; set; }
    public int Existencia { get; set; }
}

public class AgroStockContext : DbContext
```

```
{
    public DbSet<Producto> Productos => Set<Producto>();
    public DbSet<Categoria> Categorias => Set<Categoria>();

    protected override void OnConfiguring(DbContextOptionsBuilder o) =>
        o.UseSqlite("Data Source=agrostock_ef.db");

    protected override void OnModelCreating(ModelBuilder mb)
    {
        mb.Entity<Producto>().HasIndex(p => p.Codigo).IsUnique();
        mb.Entity<Categoria>().HasData(new Categoria { Id = 1, Nombre = "Fertilizantes" });
    }
}
```

El DbContext representa una sesión de trabajo con la base: rastrea los objetos que se agregan o modifican y, al llamar a SaveChanges(), genera y ejecuta los INSERT, UPDATE y DELETE necesarios dentro de una transacción. Database.EnsureCreated() crea la base y las tablas si no existen; en proyectos mayores se usan migraciones (dotnet ef migrations add).

Unidad IV · 3. CRUD con LINQ sin escribir SQL

Operación	EF Core	SQL generado (aprox.)
Read	db.Productos.Where(p => p.Existencia < 10).ToList()	SELECT … FROM Productos WHERE Existencia < 10
Create	db.Productos.Add(p); db.SaveChanges();	INSERT INTO Productos …
Update	p.Precio = 1500; db.SaveChanges();	UPDATE Productos SET Precio = … WHERE Id = …
Delete	db.Productos.Remove(p); db.SaveChanges();	DELETE FROM Productos WHERE Id = …

```
using var db = new AgroStockContext();
var lista = db.Productos
    .AsNoTracking()                          // solo lectura
    .Where(p => p.Nombre.Contains(filtro))
    .OrderBy(p => p.Nombre)
    .Select(p => new ProductoVista(p.Id, p.Codigo, p.Nombre, p.Categoria!.Nombre,
                                   p.Precio, p.Existencia, p.Precio * p.Existencia))
    .ToList();                               // aquí se ejecuta la consulta
```

> Ejecución diferida: una consulta LINQ no va a la base hasta que se enumera (ToList, First, Count, foreach). Construye la consulta completa antes de materializarla.

Unidad IV · 4. Generación de reportes

Un reporte presenta datos de la base de forma organizada para tomar decisiones. Su estructura clásica: encabezado (título, fecha, filtros), detalle (filas), agrupaciones y totales, y pie (numeración). El programa pide exportar a PDF o Excel; usamos dos bibliotecas libres para uso académico: QuestPDF (PDF con API fluida en C#) y ClosedXML (Excel sin necesidad de tener Office instalado).

```
Document.Create(doc => doc.Page(page =>
{
    page.Size(PageSizes.Letter);
    page.Margin(2, Unit.Centimetre);
    page.Header().Text("AgroStock – Reporte de inventario").SemiBold().FontSize(16);
    page.Content().Table(t =>
    {
```

```
        t.ColumnsDefinition(c => { c.ConstantColumn(60); c.RelativeColumn();
c.ConstantColumn(80); });
        t.Header(h => { h.Cell().Text("Código"); h.Cell().Text("Producto"); h.Cell().Text("Valor C$"); });
        foreach (var p in datos)
        {
            t.Cell().Text(p.Codigo); t.Cell().Text(p.Nombre);
t.Cell().AlignRight().Text($"{p.ValorInventario:N2}");
        }
    });
    page.Footer().AlignCenter().Text(x => { x.Span("Página "); x.CurrentPageNumber(); });
})).GeneratePdf(ruta);
```

```
using var libro = new XLWorkbook();
var hoja = libro.Worksheets.Add("Inventario");
hoja.Cell(1, 1).Value = "Código";
// … filas …
hoja.Cell(fila, 6).FormulaA1 = $"D{fila}*E{fila}";      // fórmula viva
hoja.Cell(total, 6).FormulaA1 = $"SUM(F2:F{total - 1})";
hoja.Columns().AdjustToContents();
libro.SaveAs(ruta);
```

Buenas prácticas: generar el reporte con Task.Run para no congelar la interfaz (Unidad III), pedir la ruta con SaveFileDialog (Unidad I), usar fórmulas en Excel en lugar de valores calculados, y ofrecer abrir el archivo al terminar.

Unidad V · 5. Servicios web y HTTP

Un servicio web es una aplicación que ofrece datos o funciones a otras aplicaciones a través de la red usando el protocolo HTTP. En el estilo REST, cada recurso tiene una URL y se opera con métodos HTTP; las respuestas suelen venir en JSON (y en servicios más antiguos, en XML).

Elemento	Ejemplo
Método	GET (leer), POST (crear), PUT (reemplazar), DELETE (eliminar)
URL	https://open.er-api.com/v6/latest/USD
Código de estado	200 OK · 404 No encontrado · 429 Demasiadas solicitudes · 500 Error del servidor
Encabezados	Accept: application/json · Authorization (si requiere clave)
Cuerpo (JSON)	{ "result": "success", "base_code": "USD", "rates": { "NIO": 36.62 } }

Unidad V · 6. Clases para el consumo de servicios web

Clase	Función
HttpClient	Envía peticiones HTTP y recibe respuestas. Se crea una sola instancia para toda la aplicación.
HttpResponseMessage	Respuesta: StatusCode, encabezados y contenido; EnsureSuccessStatusCode() lanza error si no es 2xx.
System.Text.Json / ReadFromJsonAsync<T>	Convierte JSON en objetos C# (deserialización).
[JsonPropertyName]	Relaciona el nombre del campo JSON con la propiedad C#.
XDocument (LINQ to XML)	Lee respuestas en XML.

```
private static readonly HttpClient http = new() { Timeout = TimeSpan.FromSeconds(10) };

public async Task<(decimal Nio, string Fuente)> ObtenerNioPorUsdAsync()
{
   try
   {
      using var resp = await http.GetAsync("https://open.er-api.com/v6/latest/USD");
      resp.EnsureSuccessStatusCode();
      var datos = await resp.Content.ReadFromJsonAsync<RespuestaTipoCambio>();
      return (datos!.Tasas["NIO"], "open.er-api.com");
   }
   catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
   {
      // Sin Internet: usar el respaldo local
      var local =
JsonSerializer.Deserialize<RespuestaTipoCambio>(File.ReadAllText("tipo_cambio_offline.json"))!;
      return (local.Tasas["NIO"], "respaldo local");
   }
}

// XML: <tasas base="USD"><tasa moneda="NIO">36.62</tasa></tasas>
var doc = XDocument.Parse(xml);
var nio = decimal.Parse(doc.Root!.Elements("tasa")
          .First(e => (string?)e.Attribute("moneda") == "NIO").Value,
          CultureInfo.InvariantCulture);
```

La llamada es asíncrona (await): mientras llega la respuesta, la interfaz sigue respondiendo. Siempre se deben prever tres fallos: sin conexión (HttpRequestException), tiempo agotado (TaskCanceledException) y respuesta inesperada (JsonException o datos faltantes).

Unidad V · 7. Estudio de caso: tipo de cambio para valorar el inventario

El inventario se valora en córdobas; la cooperativa necesita el equivalente en dólares. AgroStock consulta un servicio público y gratuito de tipos de cambio, muestra la tasa y su fuente en la barra de estado, y la incluye en el reporte PDF. En zonas con Internet intermitente, la aplicación usa un respaldo local y lo indica claramente.

- Analiza la API antes de usarla: documentación, límites de uso, licencia y atribución requerida.
- El tipo de cambio oficial de Nicaragua lo publica el Banco Central de Nicaragua (BCN); investiga si ofrece un servicio consultable y compáralo con el servicio usado en clase.
- Otros casos posibles: consultar el clima para planificar la aplicación de fertilizantes, o códigos postales/municipios desde un servicio público.

8. Integración final: AgroStock v3

```
AgroStock.E03/
├── Data/AgroStockContext.cs     ← EF Core (entidades, relaciones, semilla)
├── Data/ProductoService.cs      ← CRUD con LINQ
├── Models/Entidades.cs
├── Services/ReporteService.cs   ← QuestPDF y ClosedXML
├── Services/TipoCambioService.cs← HttpClient + JSON/XML
└── Forms/FrmPrincipal.cs · FrmProducto.cs
tipo_cambio_offline.json
```

Ejercitación

Consultas LINQ (individual). Escribe con EF Core la consulta que:

1. Devuelva los productos de la categoría «Semillas» ordenados por nombre.
2. Cuente los productos con existencia menor que 10.
3. Obtenga el producto de mayor precio (OrderByDescending + FirstOrDefault).
4. Calcule el valor total del inventario.
5. Agrupe por categoría y devuelva cantidad de productos y valor (GroupBy).

Retos

1. Nivel 1: agrega al PDF un resumen por categoría usando ProductoService.ResumenPorCategoria().
2. Nivel 2: agrega un botón que muestre los precios en USD en la cuadrícula usando el tipo de cambio obtenido.
3. Nivel 3: consume un segundo servicio web (por ejemplo, el clima de Matagalpa) y muéstralo en la barra de estado con respaldo sin conexión.

III. Cierre y consolidación del aprendizaje

Síntesis

- Un ORM mapea clases a tablas: se trabaja con objetos y LINQ, y SaveChanges genera el SQL.
- Un reporte útil tiene encabezado, detalle, totales y pie; se genera sin congelar la interfaz.
- Consumir un servicio web es enviar una petición HTTP y deserializar la respuesta, previendo fallos.
- AgroStock integra las cinco unidades: GUI, base de datos, concurrencia, ORM/reportes y servicios web.

Aplicación (trabajo interencuentro – BOA 3)

Trabajo interencuentro de la BOA 3 (producto integrador del componente):

1. Correcciones de AgroStock v2 según la retroalimentación del corte 2.
2. AgroStock v3 final con EF Core, reportes PDF/Excel y servicio web.
3. Manual de usuario (máx. 4 páginas) y video de demostración (máx. 3 minutos).
4. Autoevaluación actitudinal y reflexión sobre el aporte de AgroStock al uso sostenible de recursos.

Complementación

- Microsoft Learn: «Introducción a EF Core» – https://learn.microsoft.com/es-es/ef/core/get-started/overview/first-app
- QuestPDF: «Getting started» – https://www.questpdf.com/quick-start.html
- ClosedXML: documentación – https://docs.closedxml.io/

- Microsoft Learn: «Realización de solicitudes HTTP con HttpClient» – https://learn.microsoft.com/es-es/dotnet/fundamentals/networking/http/httpclient
- ExchangeRate-API (servicio usado en clase) – https://www.exchangerate-api.com/docs/free
- Investiga los términos de uso del servicio antes de publicarlo en una aplicación real.

Autoevaluación

Responde sin mirar el folleto y luego verifica con la clave al final. También puedes hacerla en línea en el sitio del curso.

1. ¿Qué hace un ORM?

a) Diseña formularios
b) Traduce entre objetos del programa y tablas de la base de datos
c) Comprime la base de datos
d) Genera reportes PDF

2. En EF Core, ¿qué método envía a la base los cambios pendientes?

a) Commit()
b) SaveChanges()
c) ExecuteNonQuery()
d) Update()

3. ¿Cuándo se ejecuta realmente una consulta LINQ de EF Core?

a) Al escribir Where
b) Al enumerarla (ToList, First, foreach)
c) Al compilar
d) Al cerrar la aplicación

4. ¿Por qué el total del reporte Excel se escribe como fórmula =SUM(...)?

a) Porque es más bonito
b) Para que se recalcule si el usuario cambia un valor
c) Porque ClosedXML no admite números
d) No importa

5. ¿Qué código de estado HTTP indica éxito?

a) 200
b) 404
c) 500
d) 301

6. ¿Qué excepción aparece típicamente cuando no hay conexión a Internet al usar HttpClient?

a) NullReferenceException
b) HttpRequestException
c) FormatException
d) IndexOutOfRangeException

7. ¿Por qué se usa await al consumir un servicio web desde un formulario?

a) Para que la petición sea más rápida
b) Para esperar la respuesta sin bloquear el hilo de la interfaz

c) Porque HttpClient no funciona sin await
d) Para cifrar los datos

Lista de verificación: marca lo que ya lograste.

- ☐ Configuré AgroStockContext con entidades y datos semilla
- ☐ Hice el CRUD con LINQ y SaveChanges
- ☐ Generé el reporte PDF
- ☐ Generé el reporte Excel con fórmulas
- ☐ Consumí el servicio de tipo de cambio
- ☐ Probé la aplicación sin Internet
- ☐ Presenté y defendí AgroStock v3
- ☐ Entregué manual, video y autoevaluación

Glosario

Término	Definición
ORM	Mapeador objeto-relacional: traduce entre objetos del programa y tablas de la base.
DbContext	Clase de EF Core que representa una sesión con la base y rastrea cambios.
DbSet<T>	Colección que representa una tabla y sobre la que se escriben consultas LINQ.
LINQ	Consultas integradas en el lenguaje C# (Where, Select, OrderBy, GroupBy).
SaveChanges	Método que envía a la base todos los cambios pendientes en una transacción.
API REST	Servicio web que expone recursos por URL y se opera con métodos HTTP.
JSON	Formato de texto para intercambiar datos: objetos {}, listas [], texto, números.
Deserializar	Convertir texto JSON o XML en objetos del programa.
Código de estado HTTP	Número que indica el resultado de la petición (200, 404, 500…).

Referencias

Bibliografía del programa:

- Ceballos Sierra, F. J. Enciclopedia de Microsoft Visual C#. Editorial RA-MA.
- Deitel, H. M. y Deitel, P. J. C# Cómo programar. Editorial Pearson, Prentice Hall.

Documentación complementaria (propuesta por el docente):

- Microsoft (2024). Guía de escritorio (Windows Forms .NET). Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/
- Microsoft (2024). Microsoft.Data.Sqlite: información general. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/data/sqlite/
- Microsoft (2024). Subprocesamiento administrado y programación asincrónica. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/threading/
- Microsoft (2024). Entity Framework Core. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/
- Microsoft (2024). Realización de solicitudes HTTP con la clase HttpClient. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/fundamentals/networking/http/httpclient

- SQLite Consortium (2024). SQL As Understood By SQLite. https://www.sqlite.org/lang.html
- QuestPDF (2024). Documentación. https://www.questpdf.com/
- ClosedXML (2024). Documentación. https://docs.closedxml.io/

Clave de la autoevaluación

- 1) b – El ORM mapea clases a tablas y genera el SQL necesario.
- 2) b – SaveChanges genera los INSERT/UPDATE/DELETE y los ejecuta en una transacción.
- 3) b – Es ejecución diferida: la consulta se envía a la base cuando se materializa.
- 4) b – Una fórmula mantiene el reporte coherente al editarlo.
- 5) a – Los códigos 2xx indican éxito; 200 = OK.
- 6) b – HttpRequestException indica que la petición no pudo completarse.
- 7) b – await libera el hilo de la UI mientras llega la respuesta.

Nota: material elaborado con apoyo de inteligencia artificial para la organización y redacción inicial; revisado por el docente.